

POST MORTEM

Aplicação da Classe Pilha

Problema II - Torre de Hanói

MAE014 - Tópico em Ciência de Dados B: Estrutura de Dados

Integrantes	DRE
Alanna Figueiredo Simões	120053919
Artur de Melo Brito	117224234
Emanuel Cardoso	122033151
Felipe Rodrigues Schoeffer	123311188

Log de Iteração e Prompts

Prompt 1 - Código Bruto

No primeiro prompt foi anexado o arquivo pilhas_refatorado.ipynb, resultado do trabalho anterior, como base para utilização da classe Pilha. A partir dele foi dado o enunciado do trabalho que gerou o código bruto.

tomando como implementação da classe pilha o arquivo anexado: construa um programa que soluciona o problema da “Torre de Hanói” com N discos usando uma rotina recursiva. A Torre em questão é uma Pilha de N discos com raios crescentes [...]

```

def mostra_pinos(pino_inicial, pino_intermediario, pino_destino,
movimentos):
    print()
    print(f"Posição: {movimentos} movimentos")
    print()

    pinos = [
        pino_inicial,
        pino_intermediario,
        pino_destino
    ]

    nomes = [
        "Pino inicial",
        "Pino intermediário",
        "Pino destino"
    ]

    for nome, pino in zip(nomes, pinos):
        elementos = []

        for i in range(pino.tamanho()):
            elementos.append(str(pino.dados[i]))

        print(f"{nome}: [ {' '.join(elementos)} ]")

    print()
def mover_disco(origem, destino):
    disco = origem.desempilha()

    # Verifica se o movimento respeita a regra
    if not destino.pilha_esta_vazia():
        topo = destino.dados[destino.tamanho() - 1]

        if disco > topo:
            # Se o movimento for inválido, devolve o disco
            origem.empilha(disco)
            raise ValueError(
                "Não é permitido colocar um disco maior sobre um
menor."
            )

        destino.empilha(disco)

```

```
def hanoi(n, origem, destino, intermediario,
          movimentos, M, estado):
    if n == 0:
        return

    # Move os n-1 discos menores para o pino intermediário
    hanoi(
        n - 1,
        origem,
        intermediario,
        destino,
        movimentos,
        M,
        estado
    )

    # Move o maior disco restante para o destino
    mover_disco(origem, destino)

    estado[0] += 1

    # Mostra a situação a cada M movimentos
    if estado[0] % M == 0:
        mostra_pinos(
            origem,
            intermediario,
            destino,
            estado[0]
        )

        input("Pressione ENTER para continuar...")

    # Move os n-1 discos do intermediário para o destino
    hanoi(
        n - 1,
        intermediario,
        destino,
        origem,
        movimentos,
        M,
        estado
    )
```

```

# =====
# PROGRAMA PRINCIPAL
# =====

print("=====")
print("          TORRE DE HANÓI")
print("=====")

N = int(input("Digite a quantidade de discos N: "))

while N <= 0:
    print("N deve ser maior que zero.")
    N = int(input("Digite a quantidade de discos N: "))

M = int(input("Digite a quantidade de movimentos M entre visualizações:
"))

while M <= 0:
    print("M deve ser maior que zero.")
    M = int(input("Digite a quantidade de movimentos M entre
visualizações: "))

# Criação dos três pinos
pino_inicial = Pilha(N, int)
pino_intermediario = Pilha(N, int)
pino_destino = Pilha(N, int)

# Coloca os discos no pino inicial.
# N é o maior disco e 1 é o menor.
for disco in range(N, 0, -1):
    pino_inicial.empilha(disco)

# Apresenta a posição inicial
mostra_pinos(
    pino_inicial,
    pino_intermediario,
    pino_destino,
    0
)

```

```

input("Pressione ENTER para iniciar a solução...")

# Lista usada para manter o contador entre as chamadas recursivas.
# O valor começa em zero.
estado = [0]

# Resolve a Torre de Hanói
hanoi(
    N,
    pino_inicial,
    pino_destino,
    pino_intermediario,
    0,
    M,
    estado
)

# Caso o número total de movimentos não seja múltiplo de M,
# apresenta também a última posição.
if estado[0] % M != 0:
    mostra_pinos(
        pino_inicial,
        pino_intermediario,
        pino_destino,
        estado[0]
    )

print("=====")
print("          SOLUÇÃO FINAL")
print("=====")

mostra_pinos(
    pino_inicial,
    pino_intermediario,
    pino_destino,
    estado[0]
)

print(f"Total de movimentos: {estado[0]}")

# Quantidade teórica mínima de movimentos
print(f"Movimentos esperados: {2*N - 1}")

```

Code Review

Falhas de Eficiência

A partir do código bruto foram identificados alguns pontos a serem alterados:

1. A apresentação dos pinos não era fixa.

Durante a recursividade, os papéis de pino inicial, intermediário e destino são trocados entre as chamadas das funções. Caso a impressão utilize diretamente essas variáveis, a posição visual dos pinos poderia mudar durante a execução.

```
def mostra_pinos(pino_inicial, pino_intermediario, pino_destino,
movimentos):
    print()
    print(f"Posição: {movimentos} movimentos")
    print()

    pinos = [
        pino_inicial,
        pino_intermediario,
        pino_destino
    ]

    nomes = [
        "Pino inicial",
        "Pino intermediário",
        "Pino destino"
    ]

    for nome, pino in zip(nomes, pinos):
        elementos = []

        for i in range(pino.tamanho()):
            elementos.append(str(pino.dados[i]))

        print(f"{nome}: [ {' '.join(elementos)} ]")

    print()
```

2. Não houve tratamento de erro para entradas não inteiras nas variáveis N e M.

Entradas de outros tipos de dados, como string e ponto flutuante, provocavam um ValueError diretamente no int(input()), encerrando o programa.

```
N = int(input("Digite a quantidade de discos N: "))

while N <= 0:
    print("N deve ser maior que zero.")
    N = int(input("Digite a quantidade de discos N: "))
```

```
M = int(input("Digite a quantidade de movimentos M entre visualizações: "))

while M <= 0:
    print("M deve ser maior que zero.")
    M = int(input("Digite a quantidade de movimentos M entre visualizações: "))
```

3. Foi necessário verificar a utilização da classe Pilha.

A solução da Torre de Hanói depende diretamente de empilha() e desempilha(). Na implementação fornecida, o método desempilha() não reduzia a quantidade de elementos, apenas retornava o elemento que estava no topo, sem o excluir apenas direcionando a referência de fim da fila para o penúltimo elemento. Isso gerava uma apresentação dos pinos com elementos que não faziam parte do pino.

```
def desempilha(self):
    if self.pilha_esta_vazia():
        raise PilhaVaziaErro("A pilha está vazia.")

    # Decrementa o ponteiro do topo e retorna o elemento
    self.quantidade -= 1
    return self.dados[self.quantidade]
```

Complexidade do algoritmo

O algoritmo bruto apresenta uma complexidade de tempo exponencial, consequência do número de movimentos necessários a serem feitos na recursividade.

$$T(n) = 2^n - 1 \rightarrow O(2^n)$$

Em relação ao espaço ocupado, a maior profundidade da pilha é n , mesmo no espaço auxiliar devido à recursão, quanto ao armazenamento. Assim temos:

$$O(n)$$

Justificativa da Refatoração

1. A apresentação dos pinos não era fixa.

Na apresentação dos pinos, foi necessário manter uma ordem fixa para os três pinos, independentemente de qual deles estivesse desempenhando o papel de origem, intermediário ou destino em determinada chamada recursiva.

A recursividade da Torre de Hanói troca esses papéis durante a execução. Por isso, a função de apresentação deve receber ou acessar os pinos considerando sua posição fixa na tela, e não simplesmente a ordem dos parâmetros da função recursiva.

Dessa forma, foi utilizada uma estrutura de lista contendo os três pinos em posições fixas, permitindo que a função de impressão sempre apresente na mesma posição:

Pino inicial

Pino intermediário

Pino destino

```
def mostra_pinos(pinos, movimentos):  
  
    print(f"\nPosição: {movimentos} movimentos\n")  
  
    for pino in pinos:  
        if len(pino.dados) == 0:  
            print('[]')  
        else:  
            print(f'{pino.dados.tolist()}\n')
```

2. Não houve tratamento de erro para entradas não inteiras nas variáveis N e M.

Entradas de outros tipos de dados, como string e ponto flutuante, provocavam um `ValueError` diretamente no `int(input())`, encerrando o programa. Assim foi feito um tratamento específico para o caso para ambas as variáveis.

```
while True:
    try:
        N = int(input("Digite a quantidade de discos N: "))

        while N <= 0:
            print("N deve ser maior que zero.")
            N = int(input("Digite a quantidade de discos N: "))
        break
    except ValueError:
        print('Entrada inválida. Digite um número inteiro positivo.')
```

```
while True:
    try:
        M = int(input("Digite a quantidade de movimentos M entre visualizações: "))

        while M <= 0:
            print("M deve ser maior que zero.")
            M = int(input("Digite a quantidade de movimentos M entre visualizações: "))
        break
    except ValueError:
        print('Entrada inválida. Digite um número inteiro positivo.')
```

3. Foi necessário verificar a utilização da classe Pilha.

Também foi necessário revisar a utilização da classe Pilha para garantir que o `desempilha()` retirasse corretamente o elemento do topo e, além de retornar seu valor, atualizasse a quantidade de elementos da pilha.

```
def desempilha(self):  
    if self.pilha_esta_vazia():  
        raise PilhaVaziaErro("A pilha está vazia.")  
  
    # Decrementa o ponteiro do topo e retorna o elemento  
    self.quantidade -= 1  
    dado = self.dados[self.quantidade]  
    self.dados[self.quantidade] = 0  
  
    return dado
```

Evidência de Testes

Os mesmos testes realizados no código bruto foram realizados de forma satisfatória no código refatorado, garantindo assim a superioridade do novo código. Sendo eles:

Teste	Parâmetros analisados	Resultado Esperado
01	$N = 0$	Tratamento do valor esperado
02	$N = \text{'abc'}$	Tratamento de erro ValueError
03	$N = -1$	Tratamento do valor esperado
04	$N = 1.5$	Tratamento de erro ValueError
05	$M = 0$	Tratamento do valor esperado
06	$M = -1$	Tratamento do valor esperado
07	$M = 1.5$	Tratamento de erro ValueError
08	$M = \text{'abc'}$	Tratamento de erro ValueError
09	$N = 3$ e $M = 1$	Visualização clara dos pinos

Além disso, testes de estresse foram realizados omitindo a função de apresentação para exemplificar o crescimento exponencial do gasto de tempo do algoritmo.

N	T(N)
10	1.023
15	32.767
20	1.048.575
25	33.554.431

Análise Crítica

A I.A. produziu a primeira implementação funcional baseada na classe Pilha fornecida. O programa utiliza uma rotina recursiva para realizar os movimentos e uma função para apresentar os três pinos no terminal. Também foi implementado o parâmetro M, permitindo controlar a quantidade de movimentos executados entre cada visualização, além da utilização do ENTER para prosseguir com a execução. Mas falhou na apresentação dos pinos e na garantia de funcionamento do programa sem paradas por erro de valor.

Assim, conclui-se que a I.A. conseguiu produzir a estrutura principal da solução recursiva, mas a primeira implementação não contemplava todos os detalhes necessários para atender ao enunciado de maneira robusta. Sendo útil em acelerar a implementação inicial, mas a análise do código e a realização dos testes continuaram sendo necessárias para garantir que o programa atendessem corretamente aos requisitos do exercício.